

Seguridad en Sistemas de información

Apuntes

Edgar Tobón Sosa

2026-05-18

Tabla de contenidos

1	Tarea 1	2
1.1	Reconocimiento del sistema	2
1.2	/boot	2
1.2.1	vmlinuz-	2
1.2.2	El sistema de archivos Temporal: initrd.img-*	2
1.2.3	La configuración de compilación: config-*	3
1.2.4	Mapa de símbolos: System.map-*	3
1.2.5	Directorio de arranque grub y efi	3
1.2.6	Resumen del proceso	3
1.3	ISO	3
1.4	GRUB	4

1. Tarea 1

1.1. Reconocimiento del sistema

El comando `uname` (Unix Name) se utiliza para imprimir información detallada sobre la arquitectura del sistema, el nombre del host y el kernel de linux que se esta ejecutando.

El flag `-a` (o `--all`) le indica al comando que imprima **toda** la información disponible en un orden específico.

```
$ uname -a
```

```
Linux etoboncs-dell 6.12.86-1 #1 SMP PREEMPT_DYNAMIC Debian 6.12.86-1 (2026-05-08)
x86_64 GNU/Linux
```

El kernel se encuentra el `/boot`

```
$ /boot/
```

1.2. /boot

La carpeta `/boot` es una de las partes más críticas del Estandar de Jerarquía de archivos (FHS) de Linux. Su propósito es exclusivamente **almacenar todo lo necesario para que la computadora pueda encender e iniciar el sistema operativo** antes de que el resto del disco duro sea montado.

Como notarás el comando `ls` contiene dos archivos duplicados `6.12.73` y `6.12.86`. Esto es un comportamiento normal de Debian; cuando el sistema operativo se actualiza, conserva el kernel anterior como respaldo por si el nuevo llega a fallar.

1.2.1. vmlinuz-

Es el núcleo de Linux compilado y ejecutable. Es el corazón de sistema operativo que toma el control de hardware.

La `v` viene de virtual Memory (soporte para memoria virtual) y `linux` termina en `z` porque el binario esta comprimido (generalmente con `gzip` o `xz`) para ocupar menos espacio y cargarse más rápido en la memoria RAM al encender.

1.2.2. El sistema de archivos Temporal: `initrd.img-*`

Significa el intial RAM Disk (Disco RAM inicial).

El kernel por si solo es pequeño y no sabe como leer discos duros complejos. Este archivo es un mini-sistema operativo temporal que el cargador de arranque descomprime directamente en la memoria RAM. Contiene módulos y drivers esenciales para que le kernel puedan “enteder” tu disco duro físico, montar la verdadera partición raiz de tu Debian y luego auto-destruirse.

1.2.3. La configuración de compilación: config-*

Es un archivo de texto plano que guarda exactamente todas las opciones y parámetros con los que fue compilado ese kernel por los desarrolladores de Debian.

1.2.4. Mapa de símbolos: System.map-*

Es una tabla de búsqueda (un diccionario) en texto plano que asocia los nombres de las funciones del código fuente del kernel con sus direcciones de memoria exacta.

El kernel en ejecución no entiende nombres como `sys_open()`, solo entiende direcciones hexadecimales. Si el sistema sufre un fallo catastrófico (un kernel panic o un Oops), el sistema usa este mapa para traducir los códigos hexadecimales de pantalla a nombres de funciones legibles humanas, permitiendo saber exactamente qué función de software causó el choque.

1.2.5. Directorio de arranque grub y efi

- **grub/**: Contiene la configuración, los módulos y los temas del cargador de arranque GRUB (Grand Unified Bootloader). aquí reside el famoso archivo **grub.cfg**. Es el programa que muestra el menú en pantalla al encender la computadora para elegir qué sistema operativo arrancar.
- **efi/**: Es el punto de montaje de la partición ESP (EFI System Partition). Es un puente entre el firmware UEFI de tu tarjeta madre y el sistema operativo. Contiene los archivos ejecutables .efi firmados que la placa base ejecuta antes de darle control a GRUB.

1.2.6. Resumen del proceso

Cuando enciendes tu computadora, la **UEFI** busca en `/boot/efi`, este levanta al **GRUB** (ubicado en `/boot/grub`), GRUB extrae el kernel **vmlinuz** y el **initrd.img** hacia la memoria RAM, y el kernel inicia apoyándose temporalmente en el **initrd** hasta que logra tomar el control total de Debian.

1.3. ISO

Crearemos un ISO, el cual contendrá la organización que tenemos:

```
$ mkdir iso
$ cd iso

$ mkdir boot
$ cd boot

cp /boot/vmlinuz-6.12.86+deb13-amd64 .
```

1.4. GRUB

Para arrancar el sistema haremos uso del **grub-mkrescue**, por lo que necesitamos encontrar primero donde se encuentra:

```
$ which grub-mkrescue  
  
/usr/bin/grub-mkrescue
```

Para el archivo de salida del ejecutable grub-mkrescue lo guardaremos en un archivo con nombre `haceImagen.sh`

```
$ vi haceImage.iso
```

Debido a que los nuevos arrancadores utilizan el estándar UEFI en lugar del antiguo sistema BIOS, no existe la carpeta `i386-pc` y en su lugar está `x86_64-efi`.

Para poder generar una imagen ISO compatible con computadoras viejas que usan BIOS, puedes descargar los archivos ejecutando el siguiente comando

```
$ sudo apt update  
$ sudo apt install grub-pc-bin
```

Ya que contamos con la carpeta `i386` ya podemos poner el comando en el archivo `.sh`.

```
grub-mkrescue -d /usr/lib/grub/i386-pc -o ImagenLinux.iso iso
```

Ahora copiamos el archivo `grub.cfg`

```
$ cd iso  
$ mkdir grub  
$ cp /boot/grub/grub.cfg .
```

Ahora hay que realizar cambios sobre el archivo `grub.cfg` de tal forma que quede con el siguiente contenido

```
#  
# DO NOT EDIT THIS FILE  
#  
# It is automatically generated by grub-mkconfig using templates  
# from /etc/grub.d and settings from /etc/default/grub  
#  
  
menuentry 'Mi Sistema Minimo' {
```

```
load_video
echo    'Loading Linux 6.12.86+deb13-amd64 ...'
linux   /boot/vmlinuz-6.12.86+deb13-amd64 ro quiet
# echo  'Loading initial ramdisk ...'
# initrd /boot/initrd.img-6.12.86+deb13-amd64
}
```

Ya con este cambio hay que ejecutar el archivo sh para generar la imagen Linux.

```
$ sh comando.sh
```